

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 10:

Virtual Functions and Polymorphism

Course taught by: Zubair Irshad

Courtesy of Some Slides: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Splitting Code into files
2. Protected vs private vs public
3. Virtual Functions
4. Pure Virtual Functions
5. Summary

Splitting Code into Files

Through .hpp files!



Splitting Code Into Files

- When you write and compile small programs, you can place all your code into a single source file.
- When your programs get larger or you work in a team, you will want to split your code into separate source files.
- Why split?
 - **Faster compilation:** It takes time to compile a file, and it is not efficient to wait for the compiler to keep translating code that doesn't change.
 - i.e. The compiler only recompiles files that have changed.
 - **Team collaboration:** When you work with other programmers in a team, it would be very difficult for multiple programmers to edit a single source file simultaneously.



The Header File

- When a program spans multiple files, some files need to **share definitions** of classes, constants, or functions.
- There must be a path of communication between the files.
- C++ enables this communication through **header files**.
- **Header file(.h or .hpp)**: A file that tells the compiler which **classes, constants, or functions** exist in another module or library.



Header and Source Files

- A **header (.h)** file contains
 - Definitions of **classes**.
 - Definitions of **constants**.
 - **Declarations** of **functions**.
- The **source (.cpp)** file contains
 - Definitions of **member and non-member functions**.
- **Example:**
 - **cashregister.h** → declares class **CashRegister**
 - **cashregister.cpp** → implements class functions
 - **registerdemo2.cpp** → contains **main()** that uses the class



Header Guards

- When multiple cpp files include or share the same header, we must prevent duplicate inclusion.

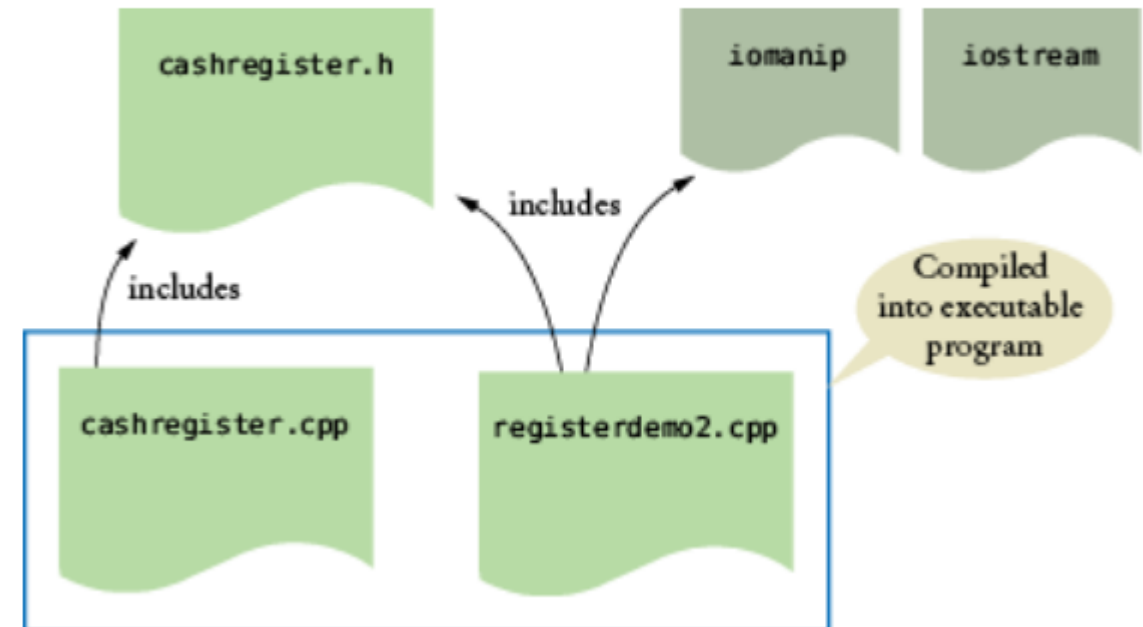
```
#ifndef CASHREGISTER_H
#define CASHREGISTER_H
// Class definition here
#endif
```

- **Why?**
 - Avoids “*redefinition*” compiler errors.
 - Ensures each header is included only once in a compilation unit.

CashRegister Example (Snippets)

```
#ifndef CASHREGISTER_H
#define CASHREGISTER_H
. . .
#endif
```

Figure 9.15.1: Compiling a Program from Multiple Source Files.





CashRegister Example

- **cashregister.cpp → cashregister.h**
 - The .cpp file **implements** the member functions of the CashRegister class.
 - It includes the .h file so it knows the **class definition** (what functions and data members exist). `#include "cashregister.h"`
- **registerdemo2.cpp → cashregister.h**
 - The main program uses the CashRegister class.
 - It also includes the header to know what the class looks like: `#include "cashregister.h"`
- **Arrows to <iostream> and <iomanip>**
 - Both cashregister.cpp and registerdemo2.cpp use standard C++ input/output.
 - `#include <iostream>`
 - `#include <iomanip>`
 - Those arrows mean these standard library headers are **included** by the source files.

Private vs Protected Keyword

How are these relevant in Inheritance?



Keyword	Accessible By	Accessible In Derived Classes?	Typical Use
private	Only within the same class	No	Hide internal data and helper functions
protected	Same class and derived classes	Yes	Allow inheritance but still hide from outside code
public	Anywhere (class, derived class, or other code)	Yes	Provide a public interface to users

Key idea:

- protected members are **inherited**, so subclasses can use them.
- private members are **not inherited** — derived classes can't access them directly.
- public inheritance keeps public and protected access levels visible to derived classes.



Virtual Functions

Prelude to Polymorphism

C++ Virtual Functions -- Motivation

- A derived class can define a function with the same name as one in its base class.
- This is called **function overriding** (or **shadowing**, if not declared virtual).
- Let's look at an example.
- Both classes define **print()**.
If we create a derived object directly, the output is 'Derived Function'

```
class Base {  
public:  
    void print() {  
        cout << "Base Function" << endl;  
    }  
};  
  
class Derived : public Base {  
public:  
    void print() {  
        cout << "Derived Function" << endl;  
    }  
};
```



The Problem: Static Binding

- What is the output of this?

```
class Base {
public:
    void print() { cout << "Base Function" << endl; }
};

class Derived : public Base {
public:
    void print() { cout << "Derived Function" << endl; }
};

int main() {
    Derived derived1;
    Base* base1 = &derived1;

    base1->print();    // What happens?
    return 0;
}
```



The Problem: Static Binding

- **What is the output of this?**
- Answer: Base Function
- **Why?**
- Things are always interesting with pointers:)
- Even though base1 points to a Derived object, the **Base version** of print() runs.
- This happens because C++ uses **static binding** (compile-time decision).
- The compiler looks at the **type of the pointer** (Base*), not the **actual object**.
- **Key takeaway:**
Without `virtual` keyword, C++ doesn't know we want **runtime behavior**.

```
class Base {
public:
    void print() { cout << "Base Function" << endl; }
};

class Derived : public Base {
public:
    void print() { cout << "Derived Function" << endl; }
};

int main() {
    Derived derived1;
    Base* base1 = &derived1;

    base1->print();    // What happens?
    return 0;
}
```



How to Fix? Declaring a Virtual Function

- Just add `virtual` keyword before the base class function
- `virtual` tells C++ to *look up* the right function using the object's actual type at run time.
- Without `virtual`, C++ *locks in* the Base version at compile time.
- **Key Takeaway:** Declaring a base class function as `virtual` allows derived classes to **replace its behavior**.
- The compiler then ensures that calls through base class pointers or references go to the **correct derived function** at runtime.

```
class Base {
public:
    virtual void print() {
        cout << "Base Function" << endl;
    }
};

class Derived : public Base {
public:
    void print() { cout << "Derived Function" << endl; }
};

int main() {
    Base* base1 = new Derived();
    base1->print();    // ✓ Derived Function
    return 0;
}
```



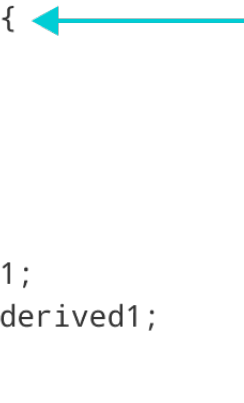

Virtual Functions: Formal Introduction

- Motivation: One application of inheritance is to work with objects whose **type** and **behavior** can vary at run time.
- This variation of behavior is achieved with ***virtual functions***.
- When you invoke a **virtual function** on an object, the C++ run-time system determines **which actual member function** to call, depending on the class to which the object belongs.
- **Definition of virtual function:** ***A function that can be redefined in a derived class. The actual function called depends on the type of the object on which it is invoked at run time.***



Virtual Functions Example

```
class Base {  
    public:  
        virtual void print() {  
            // code  
        }  
};  
  
class Derived : public Base {  
    public:  
        void print() {  
            // code  
        }  
};  
  
int main() {  
    Derived derived1;  
    Base* base1 = &derived1;  
  
    base1->print();  
  
    return 0;  
}
```



print() of Derived
class is called
because print()
of Base class is
virtual



Virtual Functions

- In order to call the functions from the **actual object type**, rather than the **reference type**, we declare each such function

```
class Question
{
public:
    Question();
    void set_text(string question_text);
    void set_answer(string correct_response);
    virtual bool check_answer(string response) const;
    virtual void display() const;
private:
    . . .
};
```



Virtual Functions and Override Keyword

- The **virtual** keyword must be used in the base class.
- All **functions** with the **same name and parameter** variable types in derived classes are then automatically virtual.
- However, it is considered good practice to supply the `virtual` reserved word for the derived-class functions as well.
- The keyword '`override`' is also used *after* the declaration of the virtual function in the derived class saying that it is derived
- If `override` doesn't actually match any virtual function in the base class, the compiler will give you an **error**.



Syntax of virtual and override keywords

```
class Base {
public:
    virtual void print() {
        cout << "Base Function" << endl;
    }
};

class Derived : public Base {
public:
    void print() override { cout << "Derived Function" << endl; }
};

int main() {
    Base* base1 = new Derived();
    base1->print();    // ✅ Derived Function
    return 0;
}
```



Logistics!

- Class Quiz 03 have been graded. Feel free to discuss this during office hours this week!
- Lab Quiz 03 this Thursday. Duration 1 hour:
 - Topics: Class and Objects, Linked List
 - For practice, attempt lab component of these topics as well as all linked list examples we studied in the class.
 - Will be announced on canvas shortly!

Logistics – Project!

- Project proposal documents has been released on Canvas!
- Key dates:
 - Project proposal document due: 7th November!
 - Project progress demos: Week of 17th November!
 - Final Evaluations: Week of 1st December!



Summary so far!

- The **virtual** keyword allows polymorphic behaviors.
- Without **virtual** keyword, we have a static binding i.e. compile time decision
- **Virtual** keyword is important for compiler to decide in **run-time**, which function to call depending on the object to which it is referenced to and not its datatype.
- **virtual** is usually placed in the base class function while **override** is placed in derived class functions.
- Every **override** call must match a function that is declared **virtual** in the base class.



Polymorphism

Prelude to Polymorphism



Definition of **Polymorphism**

- **Polymorphism** is the ability of a single interface (like a base class pointer or reference) to **represent objects of different types** and **automatically call the correct function** based on the **object's actual type** at runtime.
 - Does it remind you of what we studied for the virtual keyword?
 - Indeed, these concepts are very similar and sometimes interchangeably used.



The Slicing Problem – Motivating Example

- In object oriented programming, a derived class object can be assigned to a base class type.
- Simply put this assignment `Shape s = Circle c;` is valid.
- When a derived class object is **assigned to a base class variable**, only the **base part** of the object is copied — the derived-specific members are **sliced off**.
- The Shape object receives only Shape's data members and functions.
- Any additional data or behavior from Circle is **lost**.
- This is called **object slicing**.
- Key Takeaway: If you want polymorphic behavior (using the derived version), you must **use references or pointers**, not direct objects.



A Collection of Objects – Motivating Example

- Suppose we have several objects — some of the **base class** type, others of **derived types**.
- If we try to store them directly in an array (it would be invalid/sliced):

```
Shape shapes[3] = { Circle(), Rectangle(),  
Triangle() };
```

- Can you think of a potential reasons?



A Collection of Objects – Motivating Example

- Derived objects usually have **different sizes**.
- An array of Shape objects can't handle that.
- We need a **collection of pointers** instead i.e. Store **pointers** to the objects, not the objects themselves.

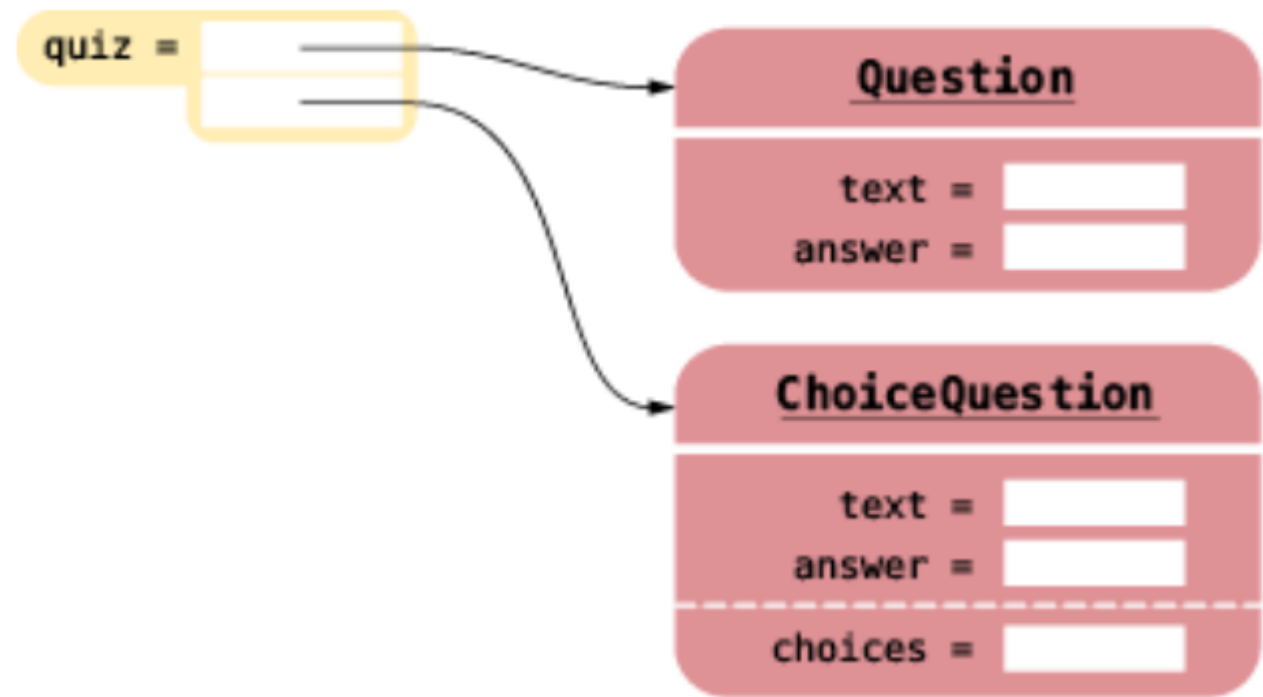
```
Shape* shapes[3];  
shapes[0] = new Circle();  
shapes[1] = new Rectangle();  
shapes[2] = new Triangle();
```

An Array of Pointers (to Objects)

Figure 10.5.2: An Array of Pointers Can Store Objects from Different Classes.

```
Question* quiz[2];  
quiz[0] = new Question();  
quiz[1] = new  
ChoiceQuestion();
```

- Each pointer in quiz points to an object — possibly of a **different derived class** while sharing a common base type.





Polymorphism – calling functions from the correct object type

- When looping through the array:

```
for (int i = 0; i < QUIZZES; i++)  
{  
    quiz[i]->display();  
    cout << "Your answer: ";  
    getline(cin, response);  
    cout << quiz[i]->check_answer(response) << endl;  
}
```

- The -> operator accesses members through the pointer.
- But how do we ensure the correct version of `display()` or `check_answer()` is called?
- That's where **polymorphism** comes in!



Polymorphism – Calling the Correct Function

- Different question types (e.g., Question, ChoiceQuestion) each have their own implementation of `display()` and `check_answer()`.

```
class Question {  
public:  
    virtual void display() const;           // virtual  
    virtual bool check_answer(string ans) const;  
};
```

```
class ChoiceQuestion : public Question {  
public:  
    void display() const override;         // override  
    bool check_answer(string ans) const override;  
};
```

- **Key takeaway:** Because these functions are **virtual** in the base class, the call `quiz[i]->display()` will automatically invoke the correct version — based on the **actual object type**.



Understanding Polymorphism

- **Polymorphism literally** = “many forms”
 - in C++, it means selecting the correct function implementation based on the **actual object type**, even when accessed through a **base class pointer or reference**.
- **Example**

```
Question* q = new ChoiceQuestion();  
q->display(); // Calls ChoiceQuestion::display()
```
- **Why It Matters**
 - Enables **flexible** and **extensible** designs.
 - Lets you treat different objects **uniformly** through a common interface.
 - Core to frameworks, UI systems, and game engines.
 - **In short:**
 - Virtual functions + inheritance = **polymorphism in action**



Polymorphism – Continued

- The quiz array collects a mixture of both kinds of questions.
- Such a collection is called polymorphic (literally, “of multiple shapes”).
- Objects in a polymorphic collection have some commonality but are not necessarily of the same type.
- Inheritance is used to express this commonality, and virtual functions enable variations in behavior.
- (Another definition of **polymorphism**: Selecting a function among several functions that have the same name on the basis of the actual type of the implicit parameter.)



Virtual Functions - Advantages

- Virtual functions give programs a great deal of **flexibility**.
- The question presentation loop describes only the general mechanism: “Display the question, get a response, and check it”.
- Each object knows on its own how to carry out the specific tasks: “Display the question” and “Check a response”.
- Using virtual functions makes programs easily **extensible**.
- One or more new classes may be added such as `NumericQuestion`.
- All that is needed is to override virtual functions in the new classes with the correct implementation.



Virtual Self Calls

- Consider the following code snippet.
- This function is in the **Question** class – and inherited by all other derived classes.

```
class Question {  
protected:  
    string text;  
    string answer;  
public:  
    void set_text(const string& t) { text = t; }  
    void set_answer(const string& a) { answer = a; }  
  
    virtual void display() const {  
        cout << text << endl;  
    }  
  
    virtual bool check_answer(const string& response) const {  
        return response == answer;  
    }  
  
    void ask() const {  
        display(); // Virtual self-call  
        cout << "Your answer: ";  
        string response;  
        getline(cin, response);  
        cout << (check_answer(response) ? "Correct!\n" : "Incorrect.\n");  
    }  
};
```



Virtual Self Calls

- Let's consider a derived class `ChoiceQuestion`
- It overrides the `display()` and `check_answer()` member functions from the main `Question` class

```
class ChoiceQuestion : public Question {
private:
    vector<string> choices;
public:
    void add_choice(const string& choice) {
        choices.push_back(choice);
    }

    void display() const override {
        Question::display();
        for (int i = 0; i < choices.size(); i++)
            cout << i + 1 << ": " << choices[i] << endl;
    }

    bool check_answer(const string& response) const override {
        int choice_num = stoi(response);
        return choices[choice_num - 1] == answer;
    }
};
```



Virtual Self Calls

```
int main(){
    ChoiceQuestion cq;
    cq.set_text("In which country was the inventor of C++ born?");
    cq.ask();
}
```

- Which `display()` and `check()` answer functions will `ask()` call?



Virtual Self Calls

```
int main(){
    ChoiceQuestion cq;
    cq.set_text("In which country was the inventor of C++ born?");
    cq.ask();
}
```

- Which `display()` and `check()` answer functions will `ask()` call?
- A. Even though `ask()` is defined in `Question`, the calls to `display()` and `check_answer()` inside it will invoke **`ChoiceQuestion::display()` and `ChoiceQuestion::check_answer()`** because those are **virtual** and overridden.



Virtual Destructors

- Base class **destructors** should always be **virtual**.
- Suppose you use **delete** with a **base class pointer** to a **derived class object** to destroy the derived-class object.
- If the base-class destructor is not virtual then delete, like a normal member function, **calls the destructor for the base class, not the destructor for the derived class**.
- This will cause only the base part of the object to be destroyed.



Logistics!

- Lab Quiz 03 tomorrow!
- Topics: LinkedList + Classes and Objects
- Focus more on LinkedList and how Classes and Objects are used in them as well as various LinkedList functions/algorithms we studied in class.
- As always, practice labs for each topic!



Recap so far!

- Polymorphism in classes is achieved through **inheritance** and **virtual** function call.
- Polymorphism means selecting a function among several functions that have the same name on the basis of the actual type of the implicit parameter.
- Polymorphism allows reusability and extensibility in C++ programming!
- Polymorphic behavior inturn allows us to store classes and obejcts through the pointers without having to write a lot of extra code!



Agenda for today!

- Practice question(s) on polymorphism.
- Virtual Functions and Abstract Classes
- Concepts of Composition, Aggregation and Association
- UML Diagrams basic



Polymorphism – Class Exercise

- Create a simple program that demonstrates **inheritance and polymorphism** using the idea of animals making sounds.
- **Requirements:**
 - Define a base class Animal with a **virtual function** speak().
 - Create two derived classes — Dog and Cat — each overriding speak().
 - In main(), make an array of Animal* pointers:
 - `Animal* zoo[3];`
 - `zoo[0] = new Animal();`
 - `zoo[1] = new Dog();`
 - `zoo[2] = new Cat();`
 - Loop through the array and call speak() on each element.



Let's move to VS Code!



Follow-up Questions

- What will the output of this program be, and why?
- What happens if we remove the keyword `virtual` from the base class?
- What if we create the array as `Animal zoo[3]` instead of `Animal* zoo[3]`?
- How can we add a new animal (e.g., `Cow`) without changing existing code?



Follow-up Questions

- **What will the output of this program be, and why?**
- A. Some generic animal sound,
- Woof!,
- Meow!
- Why? Each object's own version of speak() is called.
- This happens because speak() is declared virtual i.e. function is resolved at runtime based on the object's actual type, not pointer type. We called this **runtime polymorphism**.



Follow-up Questions

- **What happens if we remove the keyword `virtual` from the base class?**
- A. Some generic animal sound
- Some generic animal sound
- Some generic animal sound
- Why? Without `virtual`, all calls are resolved statically at compile time.
- C++ uses the pointer type (`Animal*`), not the actual object type, so all calls go to `Animal::speak()`.



Follow-up Questions

- What if we create the array as `Animal zoo[3]` instead of `Animal* zoo[3]`?
- A. It will compile, but **object slicing** occurs.
- Only the **Animal part** of Dog and Cat will be copied; the derived portions are lost.
- Result: all calls again go to `Animal::speak()`.
- This shows why we must use **pointers or references** to get polymorphic behavior.



Follow-up Questions

- **How can we add a new animal (e.g., Cow) without changing existing code?**
- **A.** Just inherit and override the speak function!

Pure Virtual Functions and Abstract Classes



Pure Virtual Functions

- Sometimes, the **base class shouldn't define behavior**, only the **interface**.
- That's where **Pure Virtual functions** are needed!
- A **pure virtual function** defines a contract — “derived classes must implement this.”
- A class with ≥ 1 pure virtual function becomes an **abstract class**.
- You **cannot instantiate** it directly.



Pure Virtual Functions

```
class Base                                //base class
{
public:
    virtual void show() = 0;              //pure virtual function
};
```

- A “pure” virtual function **does not** have an **implementation** i.e. we don’t define anything in the body of function.
- A “pure” virtual function is **abstract** – i.e. it exists only in name.
- From a syntactical point of view, a pure virtual function is declared by putting the keywords `=0` after the function declaration.



Pure Virtual Functions

Here the virtual function `show()` is declared as

```
virtual void show() = 0; // pure virtual function
```

The equal sign here has nothing to do with assignment; the value 0 is not assigned to anything. The `=0` syntax is simply how we tell the compiler that a virtual function will be pure.

Abstract Classes

- A class containing a pure virtual function is called an ***abstract class***.
- An abstract class is not **instantiated** – i.e. objects cannot be created out of an abstract class.
- An abstract serves more like an interface or a template for derived classes.
- Let's see an example on the right!

```
class Animal {
public:
    virtual void speak() const = 0; // Pure virtual
    void introduce() const {
        cout << "I am an animal. I say: ";
        speak(); // virtual self-call still works
    }
};

int main() {
    // Animal a; Error: cannot instantiate abstract class
}
```

Abstract Classes

- A derived class inherits **all of the functions** of the base class – including pure virtual functions.
- If a derived class implements pure virtual functions by **overriding** them, then **objects** of the derived class can be **instantiated**.
- Otherwise the derived class is also ***abstract***.

```
class Animal {
public:
    virtual void speak() const = 0; // Pure virtual
    void introduce() const {
        cout << "I am an animal. I say: ";
        speak(); // virtual self-call still works
    }
};

int main() {
    // Animal a; Error: cannot instantiate abstract class
}
```




Pure Virtual Functions – Example!

- Let's see an example of pure virtual functions in action in the next few slides!



Example: Pure Virtual Functions

```
class person {  
protected:  
    string name;  
public:  
    void getName() {  
        cout << "Enter name: ";  
        cin >> name;  
    }  
  
    void putName() const {  
        cout << "Name is: " << name << endl;  
    }  
  
    // Pure virtual functions (no definition here)  
    virtual void getData() = 0;  
    virtual bool isOutstanding() const = 0;  
};
```



Example: Pure Virtual Functions

```
class student : public person {  
private:  
    float gpa;  
public:  
    void getData() override {  
        getName();  
        cout << "Enter student's GPA: ";  
        cin >> gpa;  
    }  
  
    bool isOutstanding() const override {  
        return (gpa > 3.5);  
    }  
};
```



Example: Pure Virtual Functions

```
class professor : public person {  
private:  
    int numPubs;  
public:  
    void getData() override {  
        getName();  
        cout << "Enter number of professor's publications: ";  
        cin >> numPubs;  
    }  
  
    bool isOutstanding() const override {  
        return (numPubs > 10);  
    }  
};
```



Example: Pure Virtual Functions

```
int main() {
    person* persPtr[100]; // array of base class pointers
    int n = 0;
    char choice;

    do {
        cout << "Enter student or professor (s/p): ";
        cin >> choice;

        if (choice == 's')
            persPtr[n] = new student;
        else
            persPtr[n] = new professor;

        persPtr[n++]->getData();

        cout << "Enter another (y/n)? ";
        cin >> choice;
    } while (choice == 'y');
```



Example: Pure Virtual Functions

```
cout << "\n--- Results ---\n";
for (int j = 0; j < n; j++) {
    persPtr[j]->putName();
    if (persPtr[j]->isOutstanding())
        cout << "This person is outstanding.\n";
    delete persPtr[j];
}

return 0;
}
```



Virtual Functions and Polymorphism - Summary

- Virtual Functions are a useful mechanism in C++ to implement polymorphism.
- Virtual Functions begin with the keyword 'virtual'.
- A derived class is supposed to override a virtual function inherited from the base class to exhibit polymorphic behavior.
- Pure Virtual Functions do not have any implementation.
- A class containing a pure virtual function is abstract.